



# Email Spam Detection: A Hands-On End-to-End Project

---

## Project Overview

In this project, we will build an **Email Spam Detection system** using Python. The goal is to classify emails as **spam** or **not spam (ham)**. We'll use machine learning with **text vectorization techniques** and a **classification algorithm**.

---

## Tools & Libraries Used:

- pandas (data handling)
- numpy (numerical operations)
- sklearn (machine learning)
- matplotlib / seaborn (visualizations)

---

## Step 1: Load the Dataset

```
import pandas as pd

# Load the spam dataset (CSV file with 'label' and 'text')
df = pd.read_csv('spam.csv', encoding='latin-1')[['v1', 'v2']]
df.columns = ['label', 'text']
print(df.head())
```

### Explanation:

We load the dataset using pandas.

Rename the columns to label and text for simplicity.

---

## Step 2: Preprocess the Data

```
# Convert labels: spam = 1, ham = 0
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

# Check for missing values
```



```
print(df.isnull().sum())
```

### Explanation:

Labels are encoded as numbers for training.

We check for nulls — very important before ML modeling.

---

### Step 3: Exploratory Data Analysis (Optional)

```
import seaborn as sns
import matplotlib.pyplot as plt

# Plot class distribution
sns.countplot(data=df, x='label')
plt.xticks([0, 1], ['Ham', 'Spam'])
plt.title('Distribution of Ham and Spam')
plt.show()
```

### Explanation:

Helps us visually understand how balanced our dataset is.

---

### Step 4: Text Preprocessing

```
import string
import re
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer

stemmer = PorterStemmer()
stop_words = set(stopwords.words('english'))

# Function to clean each email
def preprocess(text):
    text = text.lower()
    text = re.sub(r('[^a-zA-Z]', ' '), text) # Remove numbers and punctuation
    text = text.split()
    text = [stemmer.stem(word) for word in text if word not in stop_words]
    return ' '.join(text)
```



```
# Apply preprocessing  
df['cleaned_text'] = df['text'].apply(preprocess)
```

### Explanation:

Lowercase, remove symbols, tokenize, remove stop words, and apply stemming.

This step makes data more uniform for model learning.

---

### Step 5: Vectorize the Text

```
from sklearn.feature_extraction.text import TfidfVectorizer  
  
vectorizer = TfidfVectorizer(max_features=3000)  
X = vectorizer.fit_transform(df['cleaned_text']).toarray()  
y = df['label'].values
```

### Explanation:

TF-IDF turns words into numbers (features).

We limit to 3000 words to reduce computation.

X contains features, y contains labels.

---

### Step 6: Train-Test Split

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

### Explanation:

We split the data so that we can evaluate model performance on unseen data.

---



## Step 7: Train a Model

```
from sklearn.naive_bayes import MultinomialNB
```

```
model = MultinomialNB()  
model.fit(X_train, y_train)
```

### Explanation:

Naive Bayes is a popular classifier for text data.

It's fast and works well with TF-IDF vectors.

---

## Step 8: Evaluate the Model

```
from sklearn.metrics import accuracy_score, confusion_matrix,  
classification_report
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))  
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))  
print("Classification Report:\n", classification_report(y_test, y_pred))
```

### Explanation:

We use common metrics to check how well the model performs.

Confusion matrix helps us see false positives and negatives.

---

## Step 9: Test with Custom Email

```
def predict_spam(text):  
    cleaned = preprocess(text)  
    vector = vectorizer.transform([cleaned]).toarray()  
    prediction = model.predict(vector)  
    return "Spam" if prediction[0] == 1 else "Ham"
```

# Example

```
print(predict_spam("Congratulations! You've won a free ticket to Bahamas! Click  
now!"))
```



## Explanation:

This function allows you to input any text and get a spam/ham prediction.

Real-time testing helps you understand the model practically.

---

## Conclusion:

You've built a complete **Email Spam Detection system** from loading and cleaning the data, to training and evaluating the model, and testing it with new input. This is a practical application of NLP and machine learning used by real-world email services.